

FLEETMANAGER

SOFTWARE DESIGN

1. INTRODUZIONE

1.1 Scopo del documento

Lo scopo del presente documento è descrivere le attività di testing svolte per il progetto FleetManager, sviluppato nell'ambito del corso di Ingegneria del Software A.A. 2025/26. Il documento illustra la strategia di test adottata, gli strumenti utilizzati, i casi di test implementati e i risultati ottenuti, con l'obiettivo di verificare la correttezza, l'affidabilità e la qualità del software realizzato.

Il testing rappresenta una fase fondamentale del ciclo di vita del software e consente di individuare errori, anomalie e comportamenti non conformi ai requisiti definiti nel documento di Analisi dei Requisiti. In particolare, il presente documento si concentra prevalentemente sui test di unità, implementati mediante il framework JUnit, come richiesto dalle linee guida del corso.

1.2 Ambito del testing

Le attività di testing descritte in questo documento riguardano il sistema FleetManager nella sua versione attuale e si focalizzano sulle componenti principali dell'applicazione, in particolare:

- la logica di business, implementata nel Service Layer;
- le classi di gestione delle entità principali (veicoli, utenti, prenotazioni, scadenze, manutenzioni);
- i meccanismi di validazione dei dati e gestione degli errori.

Il testing dell'interfaccia grafica (JavaFX) non è oggetto principale di questo documento, in quanto il focus è posto sulla verifica del comportamento funzionale delle classi e dei metodi che costituiscono il nucleo logico del sistema.

1.3 Riferimenti

Le attività di testing sono state condotte facendo riferimento a:

- le linee guida del corso di Ingegneria del Software;

- i principi illustrati nel testo Software Engineering: Principles and Practice di Van Vliet;
- il documento di Analisi dei Requisiti, utilizzato come base per la definizione dei casi di test;
- il framework JUnit 5 per l'implementazione dei test automatici.

1.4 Organizzazione del documento

Il documento è strutturato come segue:

- la **Sezione 2** descrive la strategia di testing adottata e gli strumenti utilizzati;
- la **Sezione 3** presenta la pianificazione delle attività di test;
- la **Sezione 4** illustra in dettaglio i test di unità implementati, organizzati per classe;
- la **Sezione 5 descrive eventuali bug individuati e le azioni correttive adottate;**
- la **Sezione 6** fornisce una valutazione complessiva dell'attività di testing;
- la **Sezione 7** conclude il documento con considerazioni finali.

2. STRATEGIA DI TESTING

2.1 Approccio generale

La strategia di testing adottata per il progetto FleetManager si basa principalmente su test di unità, affiancati da test di integrazione a livello di persistenza e servizi, eseguiti in un ambiente controllato e locale. L'obiettivo del testing è verificare il corretto comportamento delle componenti software rispetto ai requisiti funzionali definiti, assicurando la coerenza tra modello dei dati, livello di accesso ai dati (Repository/DAO) e logica di business (Service Layer). I test sono stati sviluppati e mantenuti parallelamente all'implementazione del codice, consentendo una verifica continua dell'evoluzione del sistema e riducendo il rischio di regressioni.

2.2 Tipologie di test adottate

Nel progetto FleetManager sono state adottate le seguenti tipologie di test:

Test di unità (Unit Testing)

I test di unità sono utilizzati per verificare il comportamento delle singole classi e dei relativi metodi, con particolare riferimento a:

- classi del model, che rappresentano le entità del dominio applicativo;
- classi del service layer, che implementano la logica di business del sistema.

Questi test consentono di verificare la correttezza delle operazioni fondamentali, la validazione degli input e la gestione dei casi limite.

Test di integrazione a livello Repository e Service

Accanto ai test di unità, sono stati implementati test che coinvolgono l'integrazione reale tra:

- Service Layer e Repository (DAO);
- Repository (DAO) e database embedded.

I test dei DAO non utilizzano oggetti simulati (mock), ma interagiscono direttamente con le implementazioni reali dei repository e con un database H2 in-memory, inizializzato appositamente per l'esecuzione dei test. L'utilizzo di un database in memoria consente di verificare il corretto funzionamento delle operazioni di persistenza (CRUD) in un ambiente controllato e riproducibile, evitando qualsiasi dipendenza da dati reali o persistenti. Questo approccio permette di validare l'integrazione tra repository e livello di persistenza, mantenendo l'isolamento e la ripetibilità dei test.

2.3 Livelli del sistema sottoposti a test

Model

Le classi del model sono testate tramite test di unità dedicati, finalizzati a verificare:

- la corretta inizializzazione degli oggetti;
- il funzionamento dei costruttori;
- la gestione degli attributi tramite getter e setter;
- il rispetto degli stati e dei vincoli del dominio.

Repository (DAO)

Le classi del repository sono testate attraverso test che verificano:

- inserimento, aggiornamento, recupero e cancellazione dei dati;
- corretta interazione con il database embedded;
- gestione dei casi di errore e delle condizioni limite.

Tali test rappresentano una forma di integrazione controllata, limitata al livello di persistenza, senza coinvolgimento di sistemi esterni.

Service Layer

Il service layer è testato verificando il corretto comportamento della logica applicativa, includendo:

- gestione delle prenotazioni;
- controllo delle scadenze;
- gestione delle manutenzioni;
- autenticazione e gestione degli utenti.

I test di questo livello attraversano più componenti del sistema e consentono di validare il coordinamento tra model, repository e servizi applicativi.

2.4 Ambiente di test

I test sono eseguiti in un ambiente locale e controllato, caratterizzato da:

- Java SE 17;
- JUnit 5 come framework di testing;
- Maven per la gestione delle dipendenze e del ciclo di build;
- database embedded, senza dipendenza da servizi esterni o infrastrutture di rete.

I test sono collocati nella directory `src/test/java`, in una struttura di package coerente con quella del codice applicativo, favorendo chiarezza, manutenibilità e tracciabilità tra codice e test.

2.5 Considerazioni sulla strategia adottata

La strategia di testing adottata consente di ottenere un buon equilibrio tra:

- verifica puntuale delle singole unità software;
- validazione dell'integrazione reale tra i livelli principali dell'architettura.

L'assenza di dipendenze esterne e l'utilizzo di un database embedded permettono di eseguire i test in modo ripetibile e deterministico, rendendo il testing uno strumento efficace di supporto allo sviluppo e alla qualità del software.

3. PIANIFICAZIONE DEI TEST

3.1 Obiettivi della pianificazione

La pianificazione delle attività di testing per il progetto FleetManager ha l'obiettivo di definire quando, come e con quali criteri i test vengono eseguiti, al fine di garantire la verifica sistematica del comportamento del software durante il suo sviluppo. Il testing è stato integrato nel ciclo di vita del progetto e utilizzato come strumento di supporto allo sviluppo, consentendo di individuare tempestivamente eventuali errori e di validare le funzionalità implementate.

3.2 Modalità di esecuzione dei test

I test sono implementati tramite il framework JUnit 5 e sono collocati nella directory `src/test/java`, secondo la struttura standard dei progetti Maven.

L'esecuzione dei test avviene:

- direttamente dall'IDE durante le fasi di sviluppo;
- tramite il ciclo di build Maven, mediante il comando `"mvn test"`, che utilizza il plugin Maven Surefire per l'esecuzione automatica dei test.

Questa modalità consente di verificare in modo sistematico il corretto funzionamento del sistema e di garantire la ripetibilità dei risultati di test.

3.3 Criteri di entrata (Entry Criteria)

Le attività di testing vengono avviate quando sono soddisfatte le seguenti condizioni:

- il codice sorgente compila correttamente senza errori;
- le classi da testare sono state implementate;
- le dipendenze del progetto sono correttamente risolte tramite Maven;
- l'ambiente di test locale è correttamente configurato.

Questi criteri garantiscono che i test vengano eseguiti su una base di codice consistente e stabile.

3.4 Criteri di uscita (Exit Criteria)

Le attività di testing sono considerate concluse quando:

- tutti i test JUnit risultano eseguiti con esito positivo;
- non sono presenti errori bloccanti nelle funzionalità testate;
- eventuali anomalie individuate durante l'esecuzione dei test sono state analizzate e risolte.

Al momento della stesura del presente documento, l'esecuzione dei test tramite `"mvn test"` ha prodotto esito positivo per l'intera suite, senza errori o fallimenti.

3.5 Organizzazione delle classi di test

Le classi di test sono organizzate in modo coerente con la struttura del codice applicativo e suddivise per livello architetturale, includendo:

- test per le classi del **model**;
- test per le classi del **repository (DAO)**;
- test per le classi del **service layer**.

Questa organizzazione facilita la tracciabilità tra codice di produzione e codice di test e contribuisce alla manutenibilità del sistema.

3.6 Considerazioni sulla pianificazione

La pianificazione adottata è proporzionata alla dimensione e alla complessità del progetto e consente di integrare efficacemente le attività di testing nel processo di sviluppo. L'esecuzione dei test sia dall'IDE sia tramite Maven permette di verificare automaticamente la correttezza del sistema e di mantenere un elevato livello di affidabilità durante l'evoluzione del software.

4. TEST DI UNITÀ

4.1 Test delle classi del Model

4.1.1 Obiettivi

I test delle classi del *model* hanno l'obiettivo di verificare il corretto comportamento delle entità che rappresentano il dominio applicativo del sistema FleetManager. In particolare, tali test mirano a garantire che le classi del model gestiscano correttamente:

- l'inizializzazione degli oggetti;
- l'impostazione e la restituzione degli attributi;
- il rispetto dei vincoli e degli stati previsti dal dominio.

I test di questo livello sono concepiti come unit test puri, in quanto non coinvolgono componenti esterne né livelli superiori dell'architettura.

4.1.2 Classe testata: Veicolo

La classe *Veicolo* rappresenta una delle entità centrali del sistema FleetManager ed è responsabile della modellazione delle informazioni relative ai veicoli gestiti dal sistema. Per questa classe è stata realizzata una classe di test dedicata:

- **Classe di test:** `VeicoloTest`
- **Package:** `it.fleetmanager.model`

4.1.3 Casi di test implementati

I test implementati nella classe `VeicoloTest` verificano i seguenti aspetti:

- corretta creazione dell'oggetto *Veicolo* tramite costruttore;
- corretta inizializzazione degli attributi principali;
- funzionamento dei metodi getter;

- corretta assegnazione e modifica dello stato del veicolo tramite i metodi di accesso.

I test non implementano logiche decisionali o regole di business, ma si concentrano sulla verifica della corretta gestione dei dati all'interno dell'entità di dominio.

4.1.4 Risultati dei test

L'esecuzione della classe `VeicoloTest` ha prodotto i seguenti risultati:

- **Numero di test eseguiti:** 4
- **Failure:** 0
- **Errori:** 0
- **Test saltati:** 0

Tutti i test relativi alla classe `Veicolo` sono stati eseguiti con esito positivo, confermando il corretto comportamento della classe rispetto ai casi di test definiti.

4.1.5 Considerazioni

I test sul model consentono di verificare la corretta rappresentazione delle entità di dominio e la coerenza dei dati gestiti dalle classi fondamentali del sistema. La validazione delle entità costituisce una base affidabile per i livelli superiori dell'architettura, che utilizzano tali oggetti per implementare la logica applicativa.

4.2 Test del Repository (DAO)

4.2.1 Obiettivi

I test del livello *repository* hanno l'obiettivo di verificare il corretto funzionamento delle classi DAO responsabili dell'accesso ai dati e della loro persistenza. In particolare, tali test mirano a validare:

- le operazioni di creazione, lettura, aggiornamento e cancellazione dei dati (CRUD);
- le query specifiche implementate dai DAO;
- la corretta integrazione tra le implementazioni reali dei repository e il livello di persistenza.

4.2.2 Ambiente di test e isolamento

I test dei DAO sono eseguiti utilizzando un **database H2 in-memory**, inizializzato appositamente per l'esecuzione dei test.

Prima di ogni singolo test, il database viene riportato a uno stato iniziale tramite la chiamata al metodo: `DatabaseTestUtils.resetDatabase()`; eseguita all'interno del metodo `@BeforeEach`.

Questo approccio garantisce che:

- ogni test sia completamente isolato dagli altri;
- non vi sia persistenza dei dati tra test successivi;
- i risultati siano **ripetibili e deterministici**;

- non vengano utilizzati dati reali.

Le implementazioni testate sono quelle reali (*DAOImpl) e non vengono utilizzati oggetti simulati (mock).

4.2.3 DAO testati

Per il livello repository sono state implementate classi di test dedicate per ciascun DAO principale del sistema FleetManager:

- VeicoloDAOImplTest
- UtenteDAOImplTest
- PrenotazioneDAOImplTest
- ScadenzaDAOImplTest
- ManutenzioneDAOImplTest
- NotificaDAOImplTest

Ogni classe di test verifica in modo sistematico il comportamento del rispettivo DAO.

4.2.4 Test del DAO Manutenzione

La classe ManutenzioneDAOImplTest verifica il corretto funzionamento delle operazioni di persistenza relative alle manutenzioni dei veicoli. I casi di test implementati includono:

- inserimento e recupero di una manutenzione tramite identificativo;
- gestione del caso di manutenzione non esistente;
- aggiornamento dei dati;
- cancellazione di una manutenzione;
- ricerca per veicolo;
- ricerca per tipo di manutenzione;
- recupero dell'identificativo massimo;
- recupero dell'elenco completo delle manutenzioni.

Tutti i test sono eseguiti su database H2 in-memory e producono esito positivo.

4.2.5 Test del DAO Notifica

La classe NotificaDAOImplTest verifica le operazioni di persistenza e interrogazione delle notifiche. I test coprono:

- inserimento di notifiche e recupero tramite utente;
- gestione del caso di notifica non esistente;
- aggiornamento dello stato di lettura e del messaggio;
- cancellazione di notifiche;
- recupero delle notifiche per utente;
- recupero delle notifiche non lette;
- gestione del caso di ricerca per scadenza nulla.

Questo insieme di test consente di validare il corretto comportamento del DAO anche in presenza di casi limite.

4.2.6 Test del DAO Prenotazione

La classe `PrenotazioneDAOImplTest` verifica il corretto funzionamento delle operazioni relative alle prenotazioni dei veicoli. I casi di test includono:

- inserimento e recupero di prenotazioni tramite identificativo;
- gestione del caso di prenotazione non esistente;
- aggiornamento dello stato della prenotazione;
- cancellazione;
- ricerca per utente (driver);
- ricerca per veicolo;
- ricerca per stato;
- verifica dell'esistenza di prenotazioni sovrapposte in un intervallo temporale;
- recupero dell'elenco completo delle prenotazioni.

Questi test validano sia le operazioni CRUD sia le query temporali più critiche del sistema.

4.2.7 Test del DAO Scadenza

La classe `ScadenzaDAOImplTest` verifica la gestione delle scadenze associate ai veicoli. I test implementati coprono:

- inserimento e recupero di una scadenza tramite identificativo;
- gestione del caso di scadenza non esistente;
- aggiornamento dello stato di notifica;
- cancellazione;
- ricerca delle scadenze per veicolo;
- ricerca delle scadenze prossime entro una data limite;
- recupero dell'elenco completo delle scadenze ordinate per data.

4.2.8 Test del DAO Utente

La classe `UtenteDAOImplTest` verifica il corretto funzionamento delle operazioni relative agli utenti del sistema. I test includono:

- inserimento e recupero di utenti tramite identificativo;
- gestione del caso di utente non esistente;
- recupero di utenti tramite email, inclusi dati inizializzati tramite seed;
- verifica dell'esistenza di un utente tramite email;
- aggiornamento dei dati dell'utente;
- cancellazione;
- recupero dell'elenco completo degli utenti;
- recupero dell'utente con ruolo di manager.

4.2.9 Test del DAO Veicolo

La classe `VeicoloDAOImplTest` verifica la persistenza e l'interrogazione dei veicoli gestiti dal sistema. I casi di test implementati includono:

- inserimento e recupero di un veicolo tramite targa;
- aggiornamento dei dati del veicolo;
- cancellazione;
- recupero dell'elenco completo dei veicoli;
- recupero dei veicoli disponibili in un intervallo temporale, considerando la presenza di prenotazioni attive.

Per quest'ultimo caso, il test utilizza direttamente una connessione al database H2 per inserire una prenotazione e verificare il corretto comportamento della query di disponibilità.

4.2.10 Considerazioni

I test del repository forniscono una verifica approfondita del livello di persistenza del sistema, validando sia le operazioni CRUD sia le query specifiche utilizzate dalla logica applicativa. L'utilizzo di un database H2 in-memory, inizializzato prima di ogni test, garantisce isolamento, ripetibilità e assenza di dipendenze da dati reali, rendendo i test affidabili e facilmente eseguibili in qualsiasi ambiente di sviluppo.

4.3 Test del Service Layer

4.3.1 Obiettivi

I test del *service layer* hanno l'obiettivo di verificare il corretto funzionamento della logica di business del sistema `FleetManager`. A differenza dei test del livello repository, questi test:

- validano flussi applicativi completi;
- verificano l'interazione tra più DAO;
- controllano il rispetto delle regole di dominio (ruoli, stati, vincoli temporali);
- includono la generazione di notifiche come effetto collaterale delle operazioni.

4.3.2 Ambiente di test

Tutti i test del service layer utilizzano:

- database H2 in-memory, inizializzato prima di ogni test;
- implementazioni reali dei DAO (`*DAOImpl`);
- assenza totale di mock o stub.

Il reset del database viene eseguito tramite: `DatabaseTestUtils.resetDatabase()`; garantendo isolamento, ripetibilità e indipendenza dei casi di test.

4.3.3 Classi di servizio testate

Sono state definite classi di test dedicate per ciascun gestore applicativo:

- `GestoreLoginTest`
- `GestorePrenotazioniTest`
- `GestoreManutenzioniTest`
- `GestoreScadenzeTest`

Ogni classe verifica in modo sistematico le funzionalità offerte dal rispettivo servizio.

4.3.4 Test del servizio di autenticazione e gestione utenti

La classe `GestoreLoginTest` verifica il comportamento del servizio responsabile dell'autenticazione e della gestione degli utenti. Funzionalità testate:

- login con credenziali corrette;
- gestione di password errata;
- gestione di email inesistente;
- validazione di input nulli o vuoti;
- creazione di nuovi utenti;
- prevenzione della creazione di utenti con email già esistente;
- aggiornamento del profilo utente;
- eliminazione di utenti;
- recupero di utenti per email;
- recupero dell'elenco completo degli utenti.

I test verificano sia i casi di successo sia i casi di errore, garantendo la robustezza del servizio.

4.3.5 Test del servizio di gestione delle prenotazioni

La classe `GestorePrenotazioniTest` verifica la logica di business associata alla gestione delle prenotazioni dei veicoli. Funzionalità testate:

- creazione di una prenotazione da parte di un driver;
- conferma di una prenotazione da parte di un manager;
- annullamento di una prenotazione da parte del driver;
- attivazione automatica di una prenotazione;
- completamento di una prenotazione;
- recupero delle prenotazioni associate a un driver;
- recupero delle prenotazioni associate a un veicolo.

I test verificano inoltre:

- la corretta transizione degli stati della prenotazione;
- il rispetto dei ruoli utente (driver vs manager);
- l'integrazione con il sistema di notifiche.

4.3.6 Test del servizio di gestione delle manutenzioni

La classe `GestoreManutenzioniTest` verifica le operazioni di business relative alla gestione delle manutenzioni dei veicoli. Funzionalità testate:

- programmazione di una manutenzione;
- inserimento della manutenzione nel sistema;
- aggiornamento dello stato del veicolo a *IN_MANUTENZIONE*;
- chiusura di una manutenzione;
- aggiornamento dello stato del veicolo a *DISPONIBILE*;
- modifica dello stato della manutenzione a seguito della chiusura.

I test confermano la corretta integrazione tra:

- DAO delle manutenzioni;
- DAO dei veicoli;
- sistema di notifiche.

4.3.7 Test del servizio di gestione delle scadenze

La classe `GestoreScadenzeTest` verifica la gestione delle scadenze amministrative e tecniche dei veicoli. Funzionalità testate:

- controllo delle scadenze entro una data limite;
- gestione del caso in cui non siano presenti scadenze;
- marcatura di una scadenza come notificata;
- gestione del caso di scadenza inesistente;
- blocco automatico del veicolo in caso di scadenza superata;
- esecuzione del controllo periodico delle scadenze;
- generazione delle notifiche associate alle scadenze.

I test validano sia il comportamento corretto sia la gestione delle condizioni di errore.

4.3.8 Considerazioni

I test del service layer dimostrano che la logica applicativa del sistema `FleetManager`:

- è correttamente separata dal livello di persistenza;
- rispetta le regole di dominio definite;
- gestisce correttamente i flussi principali e i casi limite;
- integra in modo coerente il sistema di notifiche.

L'uso di un database in-memory e di implementazioni reali consente di testare il comportamento del sistema in condizioni molto simili a quelle di esecuzione reale, mantenendo al contempo un elevato grado di controllo e affidabilità.

4.4 Considerazioni complessive sui test e copertura

4.4.1 Strategia di test adottata

Il sistema FleetManager adotta una strategia di testing multilivello, strutturata secondo i principali strati dell'architettura:

- **Model:** verifica della correttezza delle classi di dominio e dei loro metodi di base;
- **Repository (DAO):** verifica delle operazioni di persistenza e delle query;
- **Service Layer:** verifica della logica di business e dei flussi applicativi completi.

Questa strategia consente di individuare gli errori il più possibile vicino alla loro origine, facilitando la manutenzione e l'evoluzione del sistema.

4.4.2 Isolamento e ripetibilità dei test

Tutti i test sono eseguiti in un ambiente controllato basato su database H2 in-memory. Prima di ogni test, il database viene riportato a uno stato iniziale noto tramite una procedura di reset. Questo approccio garantisce che:

- ogni test sia indipendente dagli altri;
- i risultati siano ripetibili;
- non vi sia alcuna dipendenza da dati reali o persistenti.

4.4.3 Uso di implementazioni reali

In tutti i livelli testati vengono utilizzate implementazioni reali delle classi, sia per i DAO sia per i servizi applicativi. Non vengono utilizzati oggetti simulati (mock o stub). Questa scelta progettuale consente di:

- verificare l'effettiva integrazione tra i diversi livelli dell'architettura;
- testare il comportamento del sistema in condizioni molto simili a quelle di esecuzione reale;
- individuare eventuali problemi di integrazione che non emergerebbero con test isolati tramite mock.

4.4.4 Copertura funzionale

La suite di test copre le principali funzionalità del sistema, tra cui:

- autenticazione e gestione degli utenti;
- gestione dei veicoli e del loro stato;
- prenotazioni dei veicoli e relative transizioni di stato;
- gestione delle manutenzioni;
- gestione delle scadenze amministrative;
- generazione e gestione delle notifiche.

Sono inoltre testati diversi casi limite, come:

- input nulli o non validi;
- entità inesistenti;
- operazioni non consentite in base allo stato o al ruolo dell'utente.

4.4.5 Esecuzione automatica dei test

I test sono integrati nel ciclo di build del progetto e possono essere eseguiti automaticamente tramite Maven (`mvn test`). L'esecuzione completa della suite di test produce esito positivo, senza errori o fallimenti, confermando la stabilità del sistema al momento della consegna. L'integrazione dei test nel processo di build contribuisce a:

- ridurre il rischio di regressioni;
- facilitare il controllo della qualità del software;
- supportare l'evoluzione futura del progetto.

4.4.6 Conclusione

L'approccio di testing adottato per il progetto FleetManager garantisce un buon livello di affidabilità e coerenza del sistema. La combinazione di test su più livelli, l'uso di un ambiente controllato e l'assenza di dipendenze esterne rendono la suite di test solida e facilmente manutenibile. Il testing rappresenta quindi un elemento fondamentale del processo di sviluppo del progetto, contribuendo in modo significativo alla qualità complessiva del software.

5. BUG INDIVIDUATI E AZIONE CORRETTIVE

ADOTTATE

Durante l'attività di testing del sistema FleetManager non sono stati individuati bug bloccanti o errori critici che compromettessero il corretto funzionamento delle funzionalità implementate. Tutti i test automatici sviluppati sono stati eseguiti con successo, come evidenziato dai risultati ottenuti tramite l'esecuzione del comando `Maven test`, che ha prodotto una build completata senza errori né fallimenti.

Tuttavia, l'attività di test ha permesso di individuare e gestire alcuni casi limite e aspetti progettuali che hanno richiesto particolare attenzione, pur non configurandosi come bug veri e propri. In particolare, sono emerse alcune situazioni ricorrenti legate alla gestione di entità non presenti nel database, alla validazione degli input e alla necessità di garantire l'isolamento dei test.

5.1 Gestione delle entità inesistenti

In diversi componenti del sistema (in particolare nei DAO e nei service), è stato adottato un approccio basato sull'utilizzo di oggetti sentinella per rappresentare entità non trovate nel database, ad esempio tramite identificativi impostati a `-1`. Questo comportamento è stato verificato esplicitamente attraverso test dedicati (ad esempio nei metodi `getById` o `getEmail`), consentendo di validare una gestione coerente e prevedibile dei casi di errore senza ricorrere sistematicamente al lancio di eccezioni.

5.2 Isolamento dei test e reset del database

L'utilizzo di un database H2 in memoria ha reso necessario garantire l'isolamento completo dei test, evitando interferenze tra casi di test diversi. Durante le prime esecuzioni è emersa la necessità di ripristinare lo stato iniziale del database prima di ogni test. Questo aspetto è stato risolto introducendo un meccanismo di reset automatico del database tramite la classe `DatabaseTestUtils`, invocata nel metodo `@BeforeEach` delle classi di test. Questa azione correttiva ha permesso di rendere i test indipendenti tra loro, migliorando l'affidabilità dei risultati e la ripetibilità delle esecuzioni.

5.3 Ordinamento temporale dei risultati

Alcuni test relativi a liste ordinate (ad esempio prenotazioni) hanno evidenziato la necessità di verificare esplicitamente l'ordinamento dei risultati in base a campi temporali. I test sono stati quindi progettati per controllare non solo la cardinalità delle liste restituite, ma anche il corretto ordinamento cronologico, rendendo più robusta la verifica del comportamento dei metodi di query.

6. VALUTAZIONE COMPLESSIVA DELL'ATTIVITÀ DI TESTING

L'attività di testing svolta sul progetto FleetManager può essere considerata complessivamente efficace e adeguata agli obiettivi del progetto. La strategia adottata ha coperto in modo sistematico i principali livelli dell'architettura, includendo:

- test di unità sui modelli di dominio;
- test di integrazione sui componenti di persistenza (DAO);
- test sui servizi applicativi, verificando la corretta orchestrazione della logica di business.

L'uso di test automatici basati su JUnit ha consentito di verificare in modo ripetibile il comportamento del sistema e di individuare tempestivamente eventuali regressioni durante l'evoluzione del codice. L'integrazione dei test nel processo di build Maven ha inoltre garantito un controllo continuo sulla stabilità del progetto.

I risultati ottenuti, pari a 82 test eseguiti con successo, senza errori né fallimenti, indicano un buon livello di affidabilità dell'implementazione. Pur non essendo stata misurata formalmente la copertura del codice tramite strumenti specifici, la distribuzione dei test tra i vari package e componenti suggerisce una copertura funzionale significativa delle principali funzionalità del sistema. L'approccio adottato risulta coerente con il contesto accademico del progetto e con le dimensioni del team, fornendo un equilibrio adeguato tra rigore metodologico e semplicità operativa.

7. CONCLUSIONI

Il documento di testing ha descritto l'approccio seguito per la verifica del sistema FleetManager, illustrando la strategia adottata, la pianificazione delle attività, l'implementazione dei test e l'analisi dei risultati ottenuti. L'uso di test automatici e di un database in memoria ha consentito di validare il comportamento del sistema in modo controllato e ripetibile, mantenendo l'allineamento tra requisiti, progettazione e implementazione.

L'attività di testing ha svolto un ruolo centrale nel garantire la qualità del software, supportando lo sviluppo incrementale e contribuendo alla stabilità complessiva del progetto. L'assenza di bug critici e il superamento di tutti i test implementati confermano la solidità dell'architettura e delle scelte progettuali effettuate. Nel complesso, il testing ha rappresentato uno strumento fondamentale di supporto allo sviluppo del progetto FleetManager, fornendo evidenze concrete della correttezza e dell'affidabilità del sistema realizzato.

